Quantum Science and Technology

PAPER

3-regular three-XORSAT planted solutions benchmark of classical and quantum heuristic optimizers

To cite this article: Matthew Kowalsky *et al* 2022 *Quantum Sci. Technol.* **7** 025008

Quantum Science and Technology

PAPER

3-regular three-XORSAT planted solutions benchmark of classical and quantum heuristic optimizers

RECEIVED
21 October 2021

ACCEPTED FOR PUBLICATION
19 January 2022

PUBLISHED
14 February 2022

Matthew Kowalsky[1,2,3,4], Tameem Albash[5,6], Itay Hen[1,2,7] and Daniel A Lidar[1,2,8,9,*]

[1] Department of Physics and Astronomy, University of Southern California, Los Angeles, CA 90089, United States of America
[2] Center for Quantum Information Science & Technology, University of Southern California, Los Angeles, CA 90089, United States of America
[3] USRA Research Institute for Advanced Computer Science, 615 National, Mountain View, CA 94043, United States of America
[4] Quantum AI Laboratory (QuAIL) NASA Ames Research Center, Mail Stop 269-1, Moffett Field, CA 94035, United States of America
[5] Department of Electrical and Computer Engineering, University of New Mexico, Albuquerque, NM 87131, United States of America
[6] Department of Physics and Astronomy and Center for Quantum Information and Control, CQuIC, University of New Mexico, Albuquerque, NM 87131, United States of America
[7] Information Sciences Institute, University of Southern California, Marina del Rey, CA 90292, United States of America
[8] Department of Electrical and Computer Engineering, University of Southern California, Los Angeles, CA 90089, United States of America
[9] Department of Chemistry, University of Southern California, Los Angeles, CA 90089, United States of America
[*] Author to whom any correspondence should be addressed.

E-mail: lidar@usc.edu

Keywords: quantum annealing, Ising optimizers, planted solution

Abstract

With current semiconductor technology reaching its physical limits, special-purpose hardware has emerged as an option to tackle specific computing-intensive challenges. Optimization in the form of solving quadratic unconstrained binary optimization problems, or equivalently Ising spin glasses, has been the focus of several new dedicated hardware platforms. These platforms come in many different flavors, from highly-efficient hardware implementations on digital-logic of established algorithms to proposals of analog hardware implementing new algorithms. In this work, we use a mapping of a specific class of linear equations whose solutions can be found efficiently, to a hard constraint satisfaction problem (three-regular three-XORSAT, or an Ising spin glass) with a 'golf-course' shaped energy landscape, to benchmark several of these different approaches. We perform a scaling and prefactor analysis of the performance of Fujitsu's digital annealer unit (DAU), the D-Wave advantage quantum annealer, a virtual MemComputing machine, Toshiba's simulated bifurcation machine (SBM), the SATonGPU algorithm from Bernaschi *et al*, and our implementation of parallel tempering. We identify the SATonGPU and DAU as currently having the smallest scaling exponent for this benchmark, with SATonGPU having a small scaling advantage and in addition having by far the smallest prefactor thanks to its use of massive parallelism. Our work provides an objective assessment and a snapshot of the promise and limitations of dedicated optimization hardware relative to a particular class of optimization problems.

Many problems of theoretical and practical relevance can be cast as combinatorial optimization problems over discrete configuration spaces. Of particular interest are quadratic unconstrained binary optimization (QUBO) problems, appearing in diverse fields such as machine learning, materials design, software verification, flight-traffic control, constraint satisfaction problems, portfolio management, and logistics, to name a few examples [1, 2]. These problems can be cast as finding the optimal n-bit configuration that minimizes the cost function

$$f_Q(x) = \sum_{i \leqslant j}^{n} Q_{ij} x_i x_j, \tag{1}$$

with $x_i \in \{0, 1\}$. Equivalently, this problem can be cast as finding the ground state of an Ising Hamiltonian, with the bits x_i mapped to spin variables $s_i \in \{-1, 1\}$ via $s_i = 1 - 2x_i$, after which the cost function becomes $\sum_i h_i s_i + \sum_{i<j} J_{ij} s_i s_j$. The non-zero off-diagonal elements of Q_{ij} define the weighted connectivity graph of a QUBO instance, and the diagonal elements Q_{ii} define its effective 'local fields'. Likewise, in the Ising formulation the couplings J_{ij} and local fields h_i define an Ising problem instance. QUBO is an NP-hard problem, so it is expected that all general-purpose QUBO solvers will have an exponential runtime-scaling with problem size n, making solving such problems at large n computationally intractable.

The ubiquity of QUBO problems has led to considerable effort being devoted to reducing and mitigating the computational effort needed to tackle them. For example, specialized hardware implementing FPGA-based digital annealers [3, 4], quantum annealers based on superconducting flux qubits [5–13], and coherent Ising machines based on lasers and degenerate optical parametric oscillators [14–16] have been realized. Similarly, proposals have been made for memcomputing devices that operate on terminal-agnostic self-organizing logic gates [17–19] and FPGA-based annealers that use stochastic cellular automata to perform spin updates in parallel [20] .

Published work on these novel solvers features an array of benchmarks [21–34]. Here, we perform a scaling analysis of the time-to-solution (TTS) for a class of QUBO problems designed to provide a characterization of the current and expected future performance of some of these solvers. Our benchmark problems have the advantage of a known optimal solution by construction, and the resulting mapping to QUBO retains the hardness properties of spin-glasses when solved using heuristic solvers. The QUBO solvers benchmarked here include: the virtual MemComputing machine [31], Toshiba's simulated bifurcation machine [35], Fujitsu's digital annealer unit [27], the D-Wave advantage quantum annealer [36], a SAT algorithm from Bernaschi *et al* called SATonGPU [37], and our implementation of parallel tempering [38–40].

All the solvers studied here exhibit an exponential scaling with problem size, with varying degrees of constant-factor overheads. We find small variations in the scaling exponent among most of the solvers (the exception being D-Wave, whose exponent is significantly larger), and large variations in the prefactor. Before giving the details of our findings, we describe our problem instances, metric, and briefly review the different solvers. We conclude with a discussion of our results and provide additional details on the solvers in the methods section. The appendix provides further technical details and complementary plots.

Problem description. Our benchmark problem instances are derived from $n/2$ randomly generated linear systems of equations of $n/2$ binary variables. Each equation involves three randomly chosen variables, with the constraint that a variable appears exactly in three equations. This set of equations can be solved in polynomial time via Gaussian elimination (or conjugate gradient methods [41]), so not only do we know the solution to the linear system but we can also ensure that there is a unique solution. The linear system is then mapped to a cubic unconstrained binary optimization instance (specifically a three-regular three-XORSAT instance), where each equation in the linear system corresponds to a single three-XORSAT clause. This class of problems is known to stymie heuristic annealing-type algorithms [42–47], even though the linear system can be solved efficiently. The solution to the linear system is also the solution to the optimization problem after the mapping. Finally, each instance is reduced to a QUBO [48], which inevitably introduces an overhead [29, 49]. Indeed, this final step requires the addition of $n/2$ ancillary bits (one for each clause), forming an n-bit QUBO problem. The reduction, which is depicted in figure 1, preserves the hardness of three-regular three-XORSAT, as was demonstrated in reference [48]. We refer to this class of problem instances as the 3R3X planted-solution instances. For a careful statistical mechanical analysis of this class see Bernaschi *et al* [37] and references therein. Their salient feature is a 'golf course' energy landscape, characterized by a random first order transition in the language of spin glasses. There exists an exponential (in n) number of local minima below the dynamical critical temperature ($T_\mathrm{d} > 0$), such that for temperature $T < T_\mathrm{d}$ ergodicity is broken and the equilibration timescale scales exponentially in n. In particular, this means that heuristic optimization algorithms based on random bit (spin) flipping—whether single or as clusters—are expected to take a time scaling exponentially in n to find the global minimum (ground state).

Metric description. Our objective is to quantify how the computational time to find the optimum solution scales with problem size for our suite of specialized solvers. Since the solvers we study are stochastic, we use the optimal TTS metric [21, 24]. For an ensemble of instances of size n drawn from the 3R3X problem class, the optimal TTS (optTTS) for a quantile q is the minimum time required to find the solution at least once with a desired probability of 0.99 for that quantile:

$$\langle \mathrm{TTS} \rangle_q = \min_{t_\mathrm{f}} \left\langle t_\mathrm{f} \frac{\ln\left(1 - 0.99\right)}{\ln\left[1 - p_i\left(t_\mathrm{f}\right)\right]} \right\rangle_q \frac{1}{f_\mathrm{p}(n)}. \tag{2}$$

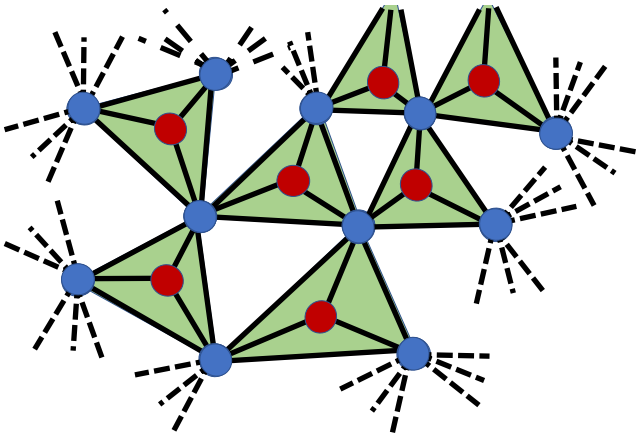

Figure 1. The connectivity graph of a three-regular three-XORSAT instance. A three-XORSAT clause (green) consists of 3 bits (blue) plus an additional auxiliary bit (red), which is used to reduce the clause locality from three-body to two-body. Each bit (or spin) appears in exactly three clauses.

Here t_f is the time the algorithm is run for, and the parallelization factor $f_p(n)$ counts how many replicas of the same instance can be run in parallel on a given solver implementation. For example, for a solver that accommodates a maximum of n_{max} bits (or spins), we have $f_p(n) = \lfloor n_{max}/n \rfloor$. Each replica r (where $r \in \{1, \ldots, f_p(n)\}$) results in a guess $E_{i,j}^{(r)}$ of the minimum (or ground state energy) of the cost function (1) for the jth run of the ith instance, and we can take $E_{i,j} = \min_r E_{i,j}^{(r)}$ as the proposed solution for that run. We assume the replicas are independent and that each has equal probability $p_i(t_f)$ of finding the optimal solution. Then, using $f_p(n)$ replicas the probability that at least one will find the ground state is $p_i'(t_f) = 1 - (1 - p_i(t_f))^{f_p(n)}$, which is also the probability that $E_{i,j}$ is the ground state energy for the jth run of the ith instance. Let R be the number of repetitions required to find the ground state at least once with probability 0.99; then $0.99 = 1 - (1 - p_i'(t_f))^R$ and $\mathrm{TTS} = t_f R$, which gives equation (2), where $\langle \cdot \rangle_q$ denotes taking the qth quantile over the distribution of instances [24].

The TTS metric balances the trade-off between a single long run of the stochastic, heuristic algorithm with a high success probability and multiple short runs with low success probability. Underlying equation (2) is the assumption that different solver runs j are independent with identical success probability for the same instance. While in principle the number of runs $R = \frac{\ln(1-0.99)}{\ln(1-p_i'(t_f))}$ should be rounded up to the nearest whole number (at least one run is necessary), we do not apply rounding since this can result in sharp changes in the TTS, which can complicate extracting the scaling behavior [24, appendix A]. We remark that it is important not to confuse the parallelization factor $f_p(n)$ with parallel execution on multiple CPU cores. Such parallelization can make the TTS as defined in equation (2) arbitrarily small and also incurs an additional energy/monetary cost that is proportional to the number of CPUs. Rather, as explained above, the parallelization factor applies when a single processor can accommodate multiple replicas.

Using the TTS metric and following the procedure outlined in methods, we calculate different quantiles of the TTS for a given problem size n for a given solver. We optimize the parameters of the solver to minimize this TTS for each size (in particular t_f), hence identifying the optTTS. This optimization is essential for avoiding the trap of feigned scaling [21], and without it an extracted scaling exponent can only be trusted as a lower bound on the true value [22]. For sufficiently large n, we then extract the scaling of optTTS as a function of n.

Finally, we remark that we chose to focus on the optTTS as defined in equation (2) since it has become the standard metric in recent years in efforts to benchmark quantum annealers against other heuristic solvers. Other metrics or methods are certainly possible and may have certain advantages; e.g., a metric based on optimal stopping theory that balances TTS with energy or monetary cost [50], or an alternative method for computing the TTS that was proposed and used for the same problem in reference [37], and argued to be less computationally demanding (see methods).

Solver descriptions. We now provide brief overviews of each benchmarked solver, with data collection, detailed algorithm descriptions, and parameter settings elaborated on in the methods section. Table 1 summarizes important properties of the solvers we benchmarked. The SATonGPU algorithm and Memcomputing solvers were benchmarked by others, as described below.

Parallel tempering (PT): parallel tempering is a Markov chain Monte Carlo method [38–40] that uses multiple replicas to speed up the equilibration dynamics. Each replica is described by a temperature and a

Table 1. Properties of the solvers used in this study, with the exception of MemComputing. Note that the couplings Q_{ij} in this study were all integer-valued; hence precision does not play a role for the digital devices but still plays a role for the analog one (D-Wave). The first four solvers were all benchmarked by the authors; the SATonGPU solver was benchmarked by Bernaschi *et al* [37].

Solver	Parallel tempering	Fujitsu digital annealer unit	Toshiba simulated bifurcation machine	D-Wave advantage 1.1	SATonGPU
Hardware	Single CPU	ASIC	Single GPU	Superconducting qubits	Single GPU
Connectivity	Full, dense	Full, dense	small n: full, dense; large n: full, sparse	Pegasus (Deg. 15)	Full, dense
Max QUBO size n	RAM-limited, 10 000	8192/4096/2048, *precision dependent	10 000 max, $10^6 J_{ij} \neq 0$	Clique: 128, 3R3X: $\approx$ 256, native: 5436	RAM-limited, >10 000
Precision	64 bit float $\approx 10^{-16}$	16/32/64 bit (signed int) $\approx 10^{-4}/10^{-9}/10^{-19}$	64 bit float $\approx 10^{-16}$	Noise-limited. $\approx$5 bit or 10^{-2}	64 bit float $\approx 10^{-16}$
Parallelization	1 per CPU core	8 per DA	40 per GPU	$\lfloor n_{\max}/n \rfloor$ replicas *connectivity dependent	327680 replicas
Accessed via	USC-UNM code	DA Center Japan 4/25/2020	Amazon Web Services [51] 8/20/2020	LEAP cloud 10/31/2020	n/a

bit/spin configuration, and each replica evolves independently using single-bit/spin Monte Carlo (MC) updates according to the Metropolis criterion [52, 53] followed by the exchange of configurations (or equivalently temperatures) between the replicas.

Our implementation of the PT algorithm uses a single CPU core, and we use it to establish a baseline against which to compare the other QUBO solvers. In practice, independent PT simulations can be run on different CPU cores to estimate the success probability of the algorithm more expediently, but we report the TTS of the algorithm using a single core. In order to predict the TTS of PT on a multi-core processor using our reported data, we need only divide the TTS reported by the number of CPU cores used. We note that further performance enhancements of PT are possible, e.g., by tuning the temperature distribution [54, 55] and recycling random numbers [56], although we did not implement such enhancements. The Houdayer [57] (or iso-energetic cluster [58]) multi-bit/spin update was tested but found not to provide a performance benefit on our 3R3X instances [48], so we did not use this type of update here.

Fujitsu digital annealer unit (DAU): Fujitsu Limited has built application-specific integrated circuits (ASICs) for performing a parallel tempering type algorithm on QUBO problems called the digital annealer [3, 4, 27]. The current generation of the DAU allows instances with up to 8192 fully connected variables to be programmed. There is a limited form of independent parallelization on the device, as it can be programmed to run 8 simultaneous replicas of 1024 fully connected variables. We use the second mode, giving the device we benchmarked a parallelization factor $f_{\mathrm{p}} = 8$ for all our instances.

The DAU implements 'rejection-free' MC updates [59] and dynamical temperature adjustment, both of which are important algorithmic differences from our parallel tempering implementation. See algorithm 3 in methods for details.

Toshiba simulated bifurcation machine (SBM): the SBM [35] uses the classical dynamics of Kerr-nonlinear parametric oscillators [28] to solve QUBO/Ising problems. We benchmarked the publicly available numerical implementation of the SBM on AWS Marketplace [51], which uses 8 Nvidia V100 Tensor Core GPUs.

D-Wave advantage: the D-Wave advantage (DWA) implements the quantum annealing algorithm [5, 60–63] in programmable hardware; it uses arrays of superconducting Josephson junctions to emulate the low energy spectrum of a time-dependent transverse field Ising Hamiltonian [7–10]. It is the only solver in our study that does not natively implement all-to-all connectivity. A minor-embedding procedure [64, 65] is necessary to fit the 3R3X instances onto the Pegasus connectivity graph of the DWA [36], increasing the physical number of variables used to implement the problem instances beyond n. See methods for more details.

Virtual MemComputing machine (MEM): our 3R3X instances were run on a virtual MemComputing machine (VMM) by a MemComputing Inc. team member, and we used the data to perform our optTTS benchmark. The VMM uses classical equations of motion to simulate the theorized digital MemComputing machine [17–19].

SATonGPU: the 'golf-course' structure of the optimization landscape, meaning that problem hardness is mainly due to entropic barriers, is well suited to a quasi-greedy algorithm [66] that is not allowed to jump over large energy barriers and does not satisfy detailed balance. Instead, a quasi-greedy algorithm performs with high probability an energy-decreasing step when possible, but when a local minimum is reached a bit/spin participating in at least one violated interaction is flipped. The algorithm chooses a random variable at each time step and flips it with a probability that depends on how many of the clauses associated with that variable are satisfied. In particular, a spin involved only in satisfied interactions is not flipped, thus ensuring that once the ground state is found, the algorithm terminates. Such an algorithm was implemented by Bernaschi *et al* [37], who in addition took advantage of massive parallelism of GPUs to run, instead of a single instance evolving for a very long time, a large number $f_{\mathrm{p}}(n) = 327\,680$ of independently evolving replicas for a short time, each one starting from a different random initial condition (see methods for a more complete description). Note that this 'SATonGPU' algorithm optimizes the original *cubic* version of the 3R3X instances, not the QUBO version that all our other solvers were given.

1. Results

We fit each solver's optTTS as a function of 3R3X instance size n to the model

$$\langle \mathrm{TTS} \rangle_q \sim 10^{\alpha n + \beta}, \tag{3}$$

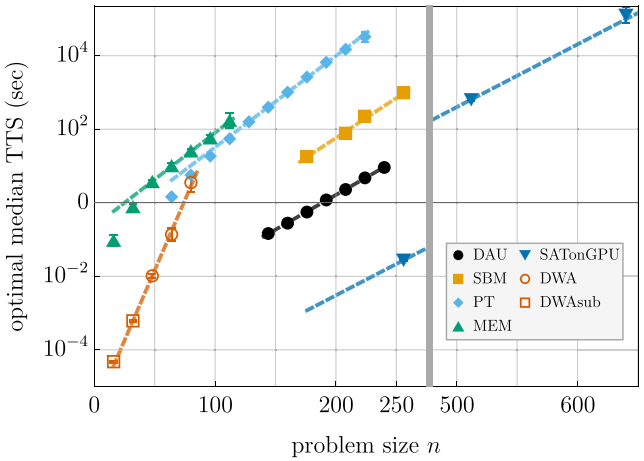

Figure 2. Median optTTS for different solvers for the quadratic 3R3X instances at different problem sizes n. The error bars correspond to 2σ confidence intervals calculated using a Bayesian-bootstrap. Each solver is represented with a different marker and color, as denoted by the legend. DAU is Fujitsu's digital annealer unit, run in parallel mode on 25 April 2020. SBM is Toshiba's simulated bifurcation machine, accessed via Amazon Web Services on 20 August 2020. PT is our implementation of parallel tempering on a single CPU core. MEM is the virtual MemComputing machine. SATonGPU is the data from figure 5 of Bernaschi *et al* [37], after converting their native three-body 3R3X results in $n/2$ variables to n two-body variables by simplying doubling their reported n values. Note that the SATonGPU results are plotted both in the left panel (ending around $n = 250$) and the right panel (starting around $n = 500$), as this solver reached significantly larger problem sizes than the other solvers; its optTTS is smaller by at least two orders of magnitude than the rest. DWA is the D-Wave advantage1.1 device accessed via LEAP on 31 October 2020. DWAsub are suboptimal points in which the optimal runtime is below 1 μs, the lowest runtime possible on the advantage1.1 device. The lines correspond to the exponential fits (equation (3)) of the data, with the coefficients given in table 2. The DWAsub points were not used in computing the DWA scaling exponent reported in table 2.

Table 2. Fit parameters α and β of equation (3) for the first quartile, median, and third quartile optTTS data corresponding to figure 2 (which shows the median only). The uncertainty on the last significant digit is indicated in parameters and reflects 2σ confidence intervals. Results are sorted by lowest α for $q = 0.5$. In all cases α was extracted after optimization of the TTS (see text), so that it reflects the true value of the scaling exponent. We did not have enough data to report the first and third quartiles of SATonGPU and DWA.

	α			β		
Solver	$q = 0.25$	$q = 0.5$	$q = 0.75$	$q = 0.25$	$q = 0.5$	$q = 0.75$
SATonGPU	n/a	0.0171(7)	n/a	n/a	−5.9(3)	n/a
DAU	0.0181(2)	0.0185(4)	0.0190(4)	−3.51(4)	−3.56(7)	−3.49(7)
SBM	0.0211(7)	0.0217(6)	0.0234(8)	−2.6(2)	−2.6(1)	−2.7(1)
PT	0.0239(1)	0.0248(2)	0.0252(1)	−0.92(2)	−0.97(4)	−0.97(2)
MEM	0.030(9)	0.025(2)	0.024(3)	−1(2)	−0.6(2)	−0.2(2)
DWA	n/a	0.08(4)	n/a	n/a	−6(2)	n/a

with the quantile $q \in \{0.25, 0.5, 0.75\}$. We find that this two-parameter model is sufficient to fit the data from each solver. We do not have enough data points to obtain a reliable fit to a more elaborate model such as $10^{\alpha n+\beta+\gamma \log n}$.

We show our main result in figure 2, where we plot the median ($q = 0.5$) optTTS of all the solvers we have benchmarked, along with the SATonGPU and MEM solvers. The scaling exponent α and the prefactor β for each solver are summarized in table 2.

As is clear from both figure 2 and table 2, the solvers separate into three groups by the scaling exponent α: (i) SATonGPU and DAU, (ii) SBM, PT and MEM, (iii) DWA. Between the two leading solvers in group (i), the SATonGPU solver has both a slightly smaller scaling exponent and by far the best absolute time to solution (see the β parameter in table 2). The latter is attributable to the massive parallelization advantage enjoyed by this solver, allowing it to run 327680 replicas (see table 1). DAU, despite running the QUBO version of the 3R3X problem, achieves nearly the same scaling as SATonGPU, but the two are separated to within 2σ statistical confidence. Notably, DAU has an advantage in both scaling and absolute time over our PT implementation, indicating that both hardware and algorithmic optimization can make a substantial difference. For example, by choosing a temperature distribution that has a constant swap probability of 0.23 between neighboring temperatures, the PT simulations of reference [66] using the original cubic form of 3R3X has a scaling that approaches that of the quasi-greedy algorithm. It is also worth noting that, as seen from table 2, the group (ii) solvers are separated to within 2σ statistical confidence from the group (i)

solvers. Finally, it is clear that DWA has the highest scaling exponent of the group, and is likewise separated from both groups (i) and (ii).

2. Discussion

We have benchmarked QUBO solvers implementing various algorithmic approaches on one particular class of problem instances. The problem class is given by a mapping of linear systems of equations representing three-regular three-XORSAT combinatorial optimization problems in $n/2$ variables to QUBO instances in n variables, and exhibits the characteristic exponential scaling with n of the solution time of NP-hard problems, due to its golf-course-like energy landscape. Furthermore, these are planted-solution problems, with the property that the solution of the linear system (which is easily found) is also the solution of the QUBO problem. The heuristic solvers we benchmarked only have access to the QUBO form of the problem (or the original three-XORSAT form in the case of SATonGPU), not the original linear system. This provides a useful class of problems against which to benchmark QUBO solvers, but we note that this problem class may not be representative of the performance of these solvers on other classes of problems. For example, the instances do not exhibit an all-to-all connectivity graph, and their precision requirements are minimal (the QUBO parameters were integers and fell in the range of $[-30, 16]$).

Only two of the solvers studied here represent dedicated hardware: DWA and DAU. The rest (PT, SBM, MEM, and SATonGPU) were benchmarked on standard digital classical hardware (CPUs or GPUs; see table 1). However, both SBM and MemComputing have been proposed as approaches that can also be implemented in dedicated hardware, and we might expect some performance improvement to accompany such an implementation. We remark that due to lack of access, our study leaves out the coherent Ising machine, which is another promising solver based on dedicated hardware [14–16].

The DWA exhibits the highest optTTS scaling in our study. Importantly, it is the only analog device among the ones we benchmarked; for all the other solvers the QUBO problem can be specified digitally within their precision range. The median minimum gap along the interpolating Hamiltonian of the three-XORSAT instances (in its native three-body form and before minor-embedding) closes exponentially as $10^{-0.0174n}$ [47]. If we assume that the adiabatic runtime scales as the inverse-gap squared (although a rigorous worst-case bound predicts an inverse-gap cubed scaling [67]), then this yields a runtime scaling exponent of $\alpha = 0.0348$ for the adiabatic algorithm. Comparing this to the α values given in table 2, it is lower than the observed DWA value. While we do not know how the mapping to the QUBO form affects the gap scaling, a difference in scaling between the adiabatic algorithm and DWA would not be surprising since the DWA does not provide a direct implementation of the adiabatic algorithm for a variety of reasons, all of which are expected to lead to reduced performance. First and foremost, it does not operate as a closed quantum system, and thermal excitation and relaxation effects usually play a detrimental role in its performance as an optimizer or sampler [24, 68–70] (with a few exceptions, e.g., reference [71]). In addition, we expect that minor-embedding and other sources of noise such as cross-talk [72] and integrated control errors [73] further hurt performance [74, 75]. While error suppression and correction can act to partly mitigate these adverse effects [76], the baseline of a large α value even in the ideal case of a closed quantum system suggests that quantum annealing is at a disadvantage in competing with state of the art classical approaches such as the ones we have explored here. However, it should be noted that the median minimum gap of the instances occurs consistently at the midpoint of the interpolating Hamiltonian, suggesting that a local adiabatic schedule could help reduce the runtime dependence to just the inverse gap [77] (i.e., $\alpha = 0.0174$), making it on-par with the best scaling observed in this study, as long as we ignore the $O(\log(n))$ due to the numerator of the adiabatic condition [78]; such a correction was also ignored in our optTTS fits for all other solvers, as mentioned above. Alternative annealing protocols could also overcome this problem and might yet prove to be beneficial [79], but are beyond the scope of the present study.

We found the Fujitsu DAU to have the lowest optTTS and scaling exponent among the solvers that solved the QUBO form of the 3R3X instances. The SATonGPU algorithm is statistically distinguishable to within 2σ confidence from the DAU in terms of optTTS scaling, and has a slightly smaller scaling exponent. The Toshiba SBM scales better than our base implementation of parallel tempering.

The SATonGPU algorithm has by far the lowest optTTS, as seen in figure 2. Recall that it solved the native three-body form of the instances, unlike all our other solvers. While locality reduction from three-variable to two-variable clauses incurs an overhead [49], and it is unclear whether avoiding this reduction provided SATonGPU a scaling exponent improvement over solving the QUBO form, what is clear is that the massive parallelization achieved by the GPU-optimized algorithm provides approximately 5 orders of magnitude reduction in the TTS. One might be tempted to interpret this as an indication of the futility of employing dedicated hardware, but we believe it instead indicates the limitations of using a single

Algorithm 1. Bayesian bootstrap for $\langle \mathrm{TTS}_i(t_\mathrm{f})\rangle_q$.

1: choose the number of bootstrap resamples $R(R = 1000)$
2: **for** $r = 1, 2, \ldots R$ **do**
3: sample I instances with replacement. Denote this set of instances by $\mathcal{I}_r$
4: Find the qth quantile of the set $\{\mathrm{TTS}_i(t_\mathrm{f})\}_{i\in\mathcal{I}_r}$ and denote it by $\langle \mathrm{TTS}_r(t_\mathrm{f})\rangle_q$
5: **end for**
6: the mean of the distribution $\left\{\langle \mathrm{TTS}_r(t_\mathrm{f})\rangle_q\right\}_{r=1}^{R}$ is taken as an estimate for $\langle \mathrm{TTS}(t_\mathrm{f})\rangle_q$ for instances from the problem class, with the standard deviation of $\left\{\langle \mathrm{TTS}_r(t_\mathrm{f})\rangle_q\right\}_{r=1}^{R}$ giving the 1σ confidence interval

problem class for general benchmarking; in this case, the relatively simple structure of the problem class allowed for the implementation of a highly optimized algorithm. Nonetheless, we hope that given its attractive properties, the 3R3X problem class will remain a useful standard for the benchmarking of both existing and future QUBO/Ising solvers and specialized devices.

Given the limited generalizability of conclusions drawn based on a single instance class, future work will employ a varied set of problem classes for benchmarking across different general-purpose solvers. We expect that different solvers will be better suited for different problem classes, and a fruitful approach will be to try a suite of solvers in attacking a given NP-hard optimization problem.

Another important future direction that our work does not address since the 3R3X instances are not well suited to it is that of *approximate* optimization. There too we expect different solvers to excel on different problem classes.

3. Methods

3.1. Bayesian bootstrap for determining optimal time to solution

In order to calculate optTTS (equation (2)), we need to estimate $p_i(t_\mathrm{f})$, the success probability of one run of a particular instance i for a given runtime t_f of the algorithm. Instead of using a sample of runs as a point estimate for $p_i(t_\mathrm{f})$, we use Bayesian inference to estimate the binomial distribution of $p_i(t_\mathrm{f})$ for each problem instance as in reference [22]. Our prior is a beta distribution $\mathrm{Beta}(\alpha, \beta)$ with $\alpha = \beta = 0.5$ (the Jeffery's prior), chosen because it is invariant to reparameterizations of the space and learns the most [80, 81]. The resulting posterior distribution for $p_i(t_\mathrm{f})$ of a particular instance is then $\mathrm{Beta}(0.5 + n_\mathrm{S}, 0.5 + N - n_\mathrm{S})$, where N is the number of runs and n_S the number of successful runs.

To estimate optTTS for all I instances (we have 100 instances for each problem size n) of a given problem size, we pick a regular distribution of t_f's, and we find the posterior distributions $\pi(p_i(t_\mathrm{f}))$ for each instance $i \in \{1, 2, \ldots, I\}$. For each instance, we can then calculate the instance TTS for the runtime t_f:

$$\mathrm{TTS}_i(t_\mathrm{f}) = t_\mathrm{f}\frac{\ln[1-0.99]}{\ln[1-p_i(t_\mathrm{f})]}\cdot\frac{1}{f_\mathrm{p}}. \tag{4}$$

We then perform a bootstrap according to algorithm 1 to determine the average of a quantile q of the TTS, $\langle \mathrm{TTS}_i(t_\mathrm{f})\rangle_q$. For each quantile q, we then define optTTS as the minimum value of the TTS attained as a function of t_f (as defined by equation (2)), which identifies a corresponding optimal value of the runtime t_f. Our definition of optTTS is over the entire set of instances of a given size and for a particular quantile and not for a single instance. If the minimum TTS is such that $\frac{\ln[1-0.99]}{\ln[1-p_i(t_\mathrm{f})]} < 1$, then we set the optimal runtime t_f^* such that $\frac{\ln[1-0.99]}{\ln[1-p_i(t_\mathrm{f}^*)]} = 1$. This corresponds to only needing to run the algorithm once.

While in our discussion above we have only explicitly optimized over the runtime t_f, optTTS and subsequent optimal scaling analysis requires all solver parameters to be properly optimized. In our discussion of each solver, we mention the extent to which we have optimized these parameters.

3.2. Parallel tempering (PT)

A single PT step includes both MC updates and replica exchanges (or replica swaps). The replicas are evolved independently by performing a fixed number of MC 'sweeps', where a single sweep corresponds to attempting to update every variable in every replica successively according to the Metropolis criterion [52, 53] for calculating the acceptance probability $\mathcal{A}$ of a bit/spin flip:

$$\mathcal{A} = \min\{1, \mathrm{e}^{-\beta\Delta E}\}, \tag{5}$$

where β is the inverse temperature of the replica, and ΔE is the change in energy associated with the spin flip.

Algorithm 2. Parallel tempering.

```
initialize replicas to random states
set replicas to log-uniformly distributed temperatures
for each MC sweep do
    for each replica (in parallel) do
        for each variable do
            propose a flip according to Metropolis criterion
            if accepted, update state and effective fields
        end for
    end for
    for each sequential pair of replicas do
        propose a replica exchange (Metropolis)
        if accepted, swap replica temperatures
    end for
end for
```

After the completion of the MC sweeps, we perform a replica swap update. The spin configuration of all neighboring replicas in temperature are swapped with a probability satisfying detailed balance:

$$\mathcal{A} = \min\{1, \mathrm{e}^{\Delta\beta\Delta E}\}, \tag{6}$$

where $\Delta\beta$ is the difference in inverse-temperatures of the two replicas being swapped, and ΔE is the difference in their energies. (Equivalently, we can hold the replica configurations fixed and swap the temperatures.)

Parameter settings. We used 32 replicas with a log-uniformly distributed (normalized to the maximum magnitude Ising parameter) inverse-temperatures β, ranging from $\beta_{\min} = 0.1$ to $\beta_{\max} = 20$. The temperature distribution was held fixed during the PT simulation. We used 10 MC sweeps between each replica swap update, as described above. Setting $\beta_{\max} = 5$ made no difference in our results.

Hardware specifications. We used Intel Xeon E5-2680 2.4 GHz with dual 10 core processors. We get a typical PT step (32 replicas, with 10 sweeps per replica, and replica swaps between all neighbor pairs) time of 1.7 μs $\times$ n.

Data collection. For each of the 100 instance within a problem size n, we ran between 10^3 and 10^4 independent PT simulations. For each simulation, we record the first time the ground state is found and terminate the simulation. Using the independent runs and their stopping times, we can estimate the cumulative distribution function for the success probability as a function of t_{f} using a Bayesian bootstrap over the runs to give error estimates. This allows us to then estimate $p_i(t_{\mathrm{f}})$ for arbitrary t_{f} values.

3.3. Fujitsu digital annealer (DAU)

A prior study by Aramon *et al* [26] benchmarked an early version of the DAU, referring to it as the parallel tempering digital annealer (PTDA), using a similar scaling analysis as ours but on different sets of instances. The early PTDA did not have replica exchanges implemented on the ASIC hardware and was performing that step on a CPU, so we expect better performance in the newer version of the DAU used in our study, where replica exchange is performed in dedicated hardware.

The DAU algorithm features several differences from the PT algorithm 2. A key difference is the implementation of 'jump' or 'rejection-free' MC Markov chains. The idea of rejection-free PT is explored in reference [82]. The key idea is to avoid the inefficiency of rejections by exploiting parallelism to compute all potential acceptance probabilities at once and accept an update move every time. Care must be taken to prevent biased estimates due to convergence to the wrong distribution, as would be the case with uniform sampling; instead, the 'jump chain' is used [82].

Algorithm. The DAU algorithm 3 replaces the serial MC sweep step from PT with a parallel calculation of every Metropolis bit/spin flip update. From the bits/spins that are proposed to be updated according to the Metropolis criterion, a single bit/spin is chosen at random and updated. This process is treated as a single MC step/iteration. With lower bounded probabilities, the process approximates the intended rejection-free roulette-wheel type selection as proven by Liposki and Lipowska [83]. However, in this implementation of the DAU, it is still possible for every spin in the parallel calculation to be rejected, resulting in repeated MC iterations with no configuration updates. To mitigate this inefficiency, after a failed configuration update, the acceptance probabilities of each variable $\mathcal{A}_j$ are artificially increased by an added energy offset E_{offset}. After the machine configuration updates, effective fields are updated in parallel, an $O(n)$ speedup relative to the same step in our parallel tempering implementation. Temperatures are actively adjusted according to a scheme by Hukushima [84] to achieve an equal replica-exchange acceptance probability for all adjacent temperatures.

Algorithm 3. Digital annealer U2 (DAU).

```
initialize replicas to random states
set replicas to log-uniformly distributed temperatures
for each MC step (iteration) do
    for each replica, in parallel do
        for each variable, in parallel do
            propose a flip using ΔE_j − E_offset
            if accepted, record
        end for
        if at least one flip accepted then
            choose one flip uniformly at random
            update states and effective fields, in parallel
            E_offset ← 0
        else
            E_offset ← E_offset + offset_increment
        end if
    end for
    if due for replica exchange then
        update temperatures
        for each sequential pair of replicas do
            propose a replica exchange (Metropolis)
            if accepted, swap replica temperatures
        end for
    end if
end for
```

Parameter settings. The 'repex_interval' parameter for the number of MC steps between replica exchanges was held fixed at 2500, the result of a coarse optimization. The 'repex_mode' parameter was set to 2, so the temperature distribution was automatically adjusted during the run without fixed temperature bounds. The number of replicas was set to the minimal value of 26. The 'offset_mode' parameter was set to 2 for automatic offset adjustment of the acceptance probabilities. The initial state of the replicas was randomized and a different random seed for the MC updates chosen for each run. Every run was performed with the DAU set at the minimal problem size of 1048 spins, and unused spins/couplings were set to maximally negative values.

Data collection. The DAU lacks a termination condition, and does not register the time to solution, only the number of MC steps to solution. To estimate $p_i(t_f)$, we repeat instance i for N times ($\approx$200) with some maximal runtime t_f^{max} and record the first time t_0 the optimal solution is reached. Then we choose a regular distribution of t_f's and update the Jeffery's prior distribution for $p_i(t_f)$ by resampling with replacement from our data. If the run we pull succeeded before (after) t_f, it counts as a success (failure), and the usual Bayesian bootstrap for $\langle \mathrm{TTS}_i(t_f)\rangle_q$ from algorithm 1 is used.

When attempting to determine the parallelization factor for the DAU, an $n = 228$ 3R3X problem was embedded 4 times into a single 1048 spin run of the DAU, with unused spins/couplings set to maximally negative values. The MC updates to solution and runtime was at least four times longer than the average MC updates to solution and runtime of the same $n = 228$ problem run alone. As such, there appears to be no parallization benefit within the same 1048 spin section. However, a single DAU has 8192 spins that can be broken into independent 1048 spin machines and operated separately. As such, in our scaling analysis for the DAU, a parallelization factor of $f_p = 8$ is used for every size we studied.

3.4. Toshiba simulated bifurcation machine

The SBM involves solving a series of coupled ordinary differential equations of the form

$$\ddot{x}_i = -(x_i^2 - p(t) + 1)x_i + C\left(h_i + \sum_j J_{ij}x_j\right), \quad x_i \in \mathbb{R} \tag{7}$$

via a symplectic Euler method. The parameter C is the relative weight of the Ising parameters $\{J_{ij}, h_i\}$, and p is steadily increased from 0 to 1 with a time step dt. The signs of the final x_i values are taken as -1 and 1 respectively to solve the Ising problem defined by $\{J_{ij}, h_i\}$. The conversion to a QUBO problem is straightforward.

Parameter settings. From equation (7), there are two parameters beyond the runtime to be optimized: C and dt—the time step taken by the symplectic Euler numerical solution to the coupled ODE's. We use the auto-adjustment feature provided for C, and manually optimize dt around a central value of 0.9 used in

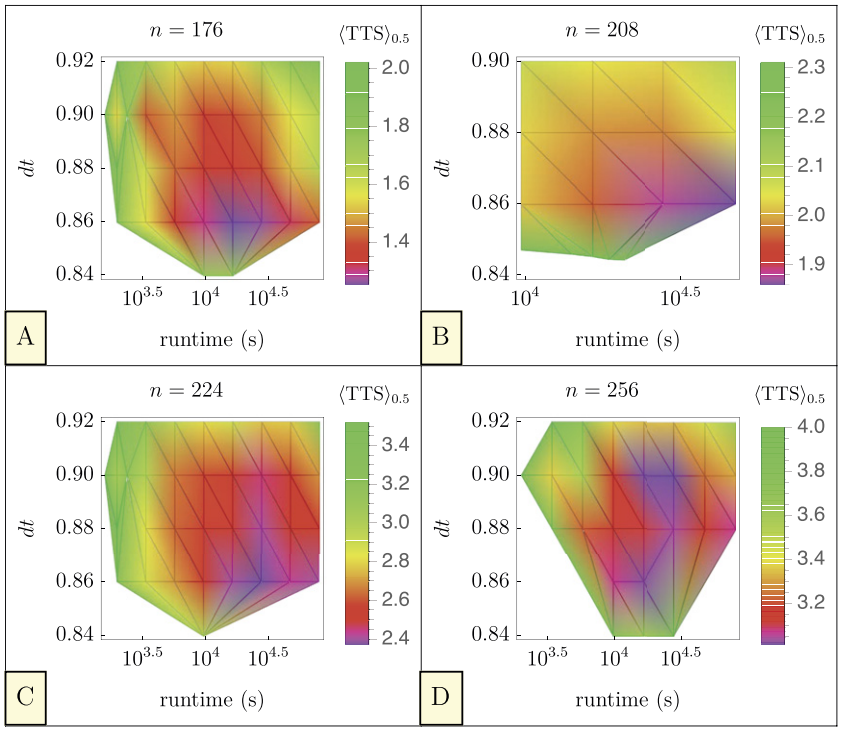

Figure 3. Plots depicting the full $\langle \mathrm{TTS} \rangle_{0.5}$ data collected on the Toshiba simulated bifurcation machine.

previous benchmarks [35]. Figure 3 depicts the resulting optimization data (without error) used to determine the median optTTS ($\langle \mathrm{TTS} \rangle_{0.5}$) for each value of n in figure 2.

Data collection. The 8 Nvidia Tensor Core GPUs run 320 replicas of the algorithm at once. Taking a single Nvidia Tensor GPU for our benchmark, we set $f_{\mathrm{p}} = 40$. The returned time t_r is precise to only 10^{-2} seconds. To resolve anneal times t_{f} below 10^{-2} seconds it is necessary to run m 'loops' of the algorithm and then take t_r/m to determine t_{f}. Runtime is not directly specifiable; instead one specifies the number of Euler steps to indirectly set runtime.

We use a log-uniform set of steps from 2000 to 80 000, with an n-dependent total time t_r from 100 s to 300 s. The number of loops m is simply the maximum attainable within t_r. Each point in the parameter mesh is repeated at least 3 times to average over variation in the auto-adjustment of C, and includes an n-dependent number of instances from 40 to 20. Extra data was collected at points with poor certainty in TTS.

3.5. D-Wave

The D-Wave processor implements a time-dependent Hamiltonian $H(s)$, where s is a dimensionless time parameter increasing from 0 to 1 in the standard 'forward' annealing mode we implemented. The Hamiltonian interpolates between the transverse-field contribution and the classical Ising term, reading

$$H(s) = -A(s)\sum_i \hat{X}_i + B(s)\left(\sum_i h_i \hat{Z}_i + \sum_{i\neq j} J_{ij}\hat{Z}_i\hat{Z}_j\right), \tag{8}$$

where $A(s)$ and $B(s)$ time-dependent functions determining the annealing schedule and are fixed by the quantum hardware. $A(s)$ decreases monotonically to 0, while $B(s)$ increases monotonically from near 0. $\{\hat{X}_i, \hat{Z}_i\}$ are the Pauli operators acting on qubit i. At the end of each annealing run ($s = 1$) the hardware returns the ± 1 eigenvalues of each $\hat{Z}_i$, which are determined by the direction of circulating persistent current in a superconducting metal loop [7, 8]. Again, the conversion to a QUBO problem is straightforward.

Data collection. Quantum annealers have a number of unique challenges to overcome in the estimation of $p_i(t_{\mathrm{f}})$. A embedding is required, introducing additional parameters that require optimization: chain strength, and chain break post-processing. In minor embedding, multiple qubits are chained together via strong ferromagnetic couplings [64, 65, 85]. For sufficiently strong ferromagnetic couplings this ensures that the ground state of the Ising Hamiltonian is unaffected by the minor embedding, but it affects the spectrum along the anneal in unpredictable ways, making the strength of these connections critical: too

high, and the logical qubit will not flip readily enough to solve the Ising problem, too low and the logical qubit will 'break' as its member physical qubits resolve into different final states that do not align [86]. With large problems and good tuning, a small number of chains still break, and it is necessary to choose how to map a broken chain to a logical state. At first, we sought to optimize all parameters on a *per size (per n)* basis: annealing time, chain strength, and embedding. Using the minorminer feature 'find_clique_embedding()' that uses heuristics from references [87, 88] to adapt the optimal embeddings to broken graphs, we found that a per n optimization via full clique embeddings at each n produced immeasurably high TTS after $n > 32$, even with the use of 'VirtualGraphComposite()', an advanced method for tuning logical qubits for a particular embedding. (This method involves correcting the flux biases of the physical qubits that are skewed due to extended-range ferromagnetic couplings used to form logical qubits.) Therefore, out of necessity but clearly giving an unfair advantage to the DWA, we instead use minorminer's heuristic algorithms [89] to find embeddings on a *per instance* basis.

The classical postprocessing chosen for all broken chains was the majority vote algorithm. Chain strength tuning at each size for $n < 80$ invariably showed that using the extended_j_range of the device to couple logical qubits together by a -2 coupling strength results in the lowest TTS. This suggests that a larger ferromagnetic chain strength would be optimal. While we have not investigated this approach in our study, one way to achieve this within the current capabilities of the hardware is to scale down the overall strength of the Ising Hamiltonian while still using a -2 coupling strength for the chain couplings. In this way, we would be able to achieve a stronger relative chain coupling. However, by reducing the overall energy scale of the problem Hamiltonian, the evolution is more susceptible to both thermal and implementation errors, so the overall rescaling of the Ising Hamiltonian would need to be optimized.

3.6. MemComputing

Simulations were performed by Dr. Fabio Traversa of MemComputing Inc. on a virtual MemComputing machine (VMM)—a software simulator which uses classical equations of motion to simulate the theorized digital MemComputing machine [17–19]. DMMs are circuits of 'self organizing logic gates' or (SOLGs) that can accept inputs from any of the gate terminals (either input or output) and self-organize their internal state so that it satisfies their logic relations. These circuits are used to solve combinatorial optimization problems by expressing the problem in Boolean format and then mapping the latter onto circuits of SOLGs.

3.7. SATonGPU

In the main text we have already described the SATonGPU algorithm in some detail, and complete details can be found in reference [37].

Parallization factor. To mitigate the algorithm's dependence on initial condition Bernaschi *et al* simulated in parallel a large number of clones (or replicas), where each clone started from a different and randomly chosen initial condition. The algorithm is considered successful when any clone reaches the solution.

A total of 327680 replicas were simulated in parallel on an Nvidia V100 GPU. All runtimes reported correspond to using this many replicas. However, we note that the parallelization factor used in reference [37] appears to be weakly n-dependent. Nevertheless, since this amounts to at most a $\log(n)$ correction to the scaling, we did not account for it explicitly, as mentioned in the main text.

TTS extraction. Reference [37] demonstrated that the TTS, which they defined as the shortest time taken by any the replicas to find the solution of a given instance, is exponentially distributed as $P[\mathrm{TTS} > \tau] = \exp(-t/\tau)$. Identifying τ is then sufficient to estimate the optTTS for SATonGPU at each size.

Acknowledgments

We are grateful to Bernaschi *et al* for sharing their data with us and to Dr Victor Martin Mayor for clarifying discussions about the SATonGPU solver, and to Dr Fabio Traversa for performing MemComputing simulations for us. We also thank Dr Tamura Hirotaka and the Fujitsu team for many interesting discussions, and Mr Yaz Izumi and the Toshiba team for feedback. We would like to acknowledge Dr Salvatore Mandra and Dr Eleanor Rieffel for useful discussions and for sharing with us their benchmarking results of a separate parallel tempering implementation, which served as a useful comparison to our own. We would also like to acknowledge Dr Aaron Lott for useful discussions concerning D-Wave devices. The research is based upon work (partially) supported by the Office of the Director of National Intelligence (ODNI), Intelligence Advanced Research Projects Activity (IARPA) and the Defense Advanced

Research Projects Agency (DARPA), via the US Army Research Office contract W911NF-17-C-0050. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the ODNI, IARPA, DARPA, or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright annotation thereon. This work is partially supported by a DOE/HEP QuantISED program Grant, QCCFP/Quantum Machine Learning and Quantum Computation Frameworks (QCCFP-QMLQCF) for HEP, Grant No. DE-SC0019219. The authors also acknowledge support by Fujitsu Limited, which in no way influenced any of the results reported here. The authors acknowledge the Center for Advanced Research Computing (CARC) at the University of Southern California for providing computing resources that have contributed to the research results reported within this publication. https://carc.usc.edu.

Data availability statement

The data that support the findings of this study are available upon reasonable request from the authors.

Appendix A. First and third quartile optTTS results

To supplement the median results shown in figure 2 in the main text, we display the first and third quartile results in figure 4. The corresponding fit parameters are given in table 2 in the main text. These results do not alter our conclusions about the relative performance of the different solvers benchmarked in this study.

Appendix B. Example of the reduction from cubic form to QUBO form

The general reduction from three-body to two-body terms is depicted graphically in figure 1 of the main text.

The cost function of an n-bit 3R3X system is a sum of n three-body terms of the form $-(-1)^{b_i}s_{i_1}s_{i_2}s_{i_3}$ defined on n Ising spins where $b_i \in \{0, 1\}$ and s_{i_1}, s_{i_2} and s_{i_3} are the spins belonging to the clause.

To reduce the locality of a clause to two- and one-body terms, we use a gadget that shares its four minimizing configurations (see reference [48] for additional details). For the negatively signed clause ($b_i = 0$), these are the four configurations whose product is positive, namely, $(1, 1, 1), (1, -1, -1)$, $(-1, 1, -1)$ and $(-1, -1, 1)$ and an appropriate gadget is

$$\begin{aligned} G_{3X} = {} & h(s_{i_1} + s_{i_2} + s_{i_3}) + \tilde{h}s_a \\ & + J(s_{i_1}s_{i_2} + s_{i_2}s_{i_3} + s_{i_3}s_{i_1}) + \tilde{J}s_a(s_{i_1} + s_{i_2} + s_{i_3}), \end{aligned} \tag{B1}$$

where $(h, \tilde{h}, J, \tilde{J})$ can be either $(-1, -2, 1, 2)$ or $(-1, 2, 1, -2)$, yielding in both cases a minimal cost of -4 (we note that other gadgets exist that can equivalently be used). The above gadget is a fully connected four-spin cost function with s_a serving as an auxiliary spin. Positively signed clauses are treated similarly.

The above gadget allows us to cast n-bit 3R3X linear systems as two-body Ising models with $2n$ spins, as each clause adds one auxiliary spin to the problem and an n-bit instance has n clauses.

Appendix C. TTS parameter fitting procedure

Our fitting procedure seeks to exclude finite size effects at small n and imperfect parameter optimization at large n. To achieve this, beginning with two representative data points for a straight line fit on a plot of $\log_{10}\langle \mathrm{TTS}\rangle_q$ vs n at intermediate n values, we successively include each larger/smaller-n data point until adding another data point to the fit starts to significantly alter α.

In order to fit the log TTS data to a linear function of the form $\alpha n + \beta$, we use Mathematica's [90] built-in function 'NonlinearModelFit' with the variance estimator function. We take as input to the function the logarithm of the TTS data and their 1σ error bars; the error bars are then treated by the function as the uncertainty in the input data points. The output of the function is the best fit estimates to the coefficients α and β with their 0.95 confidence intervals, which we report in table 2.

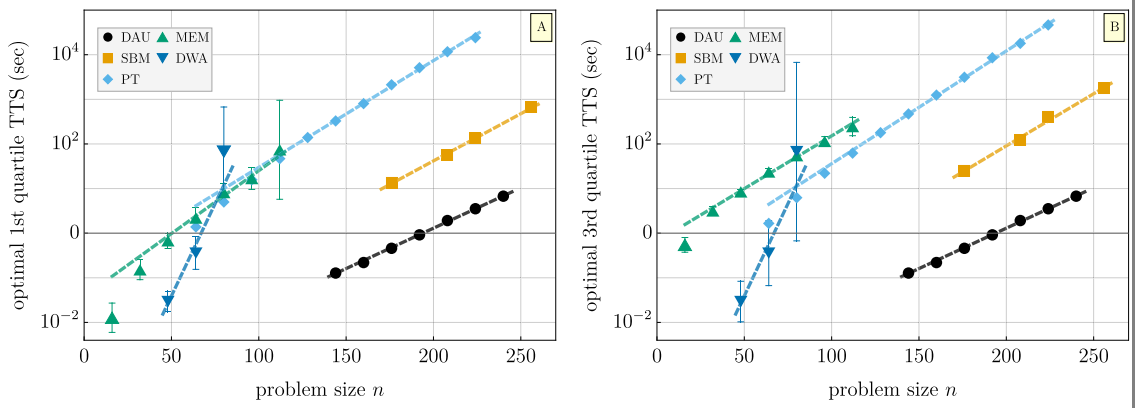

Figure 4. $\langle \mathrm{TTS} \rangle_{0.25}$ and $\langle \mathrm{TTS} \rangle_{0.75}$ results for 3R3X instances. Shown are the optTTS of the 25th percentile (A) and 75th percentile (B) hardness with 99% probability.

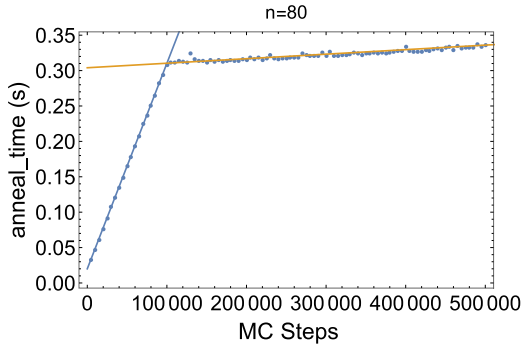

Figure 5. Extraction of the runtime per MC update t_{MC} for the Fujitsu DAU from the slope of anneal time vs number of MC steps. Shown is a representative example with $n = 80$ bits. We observe two distinct t_{MC}, which are the slopes seen in the plot. This holds regardless of n (not shown).

Appendix D. Monte Carlo time step measurement on the DAU

Every step of the core algorithmic DAU process is executed in dedicated application specific integrated circuits (ASIC) hardware, excepting the automatic parameter adjustment for the first 10^5 MC steps. Each hardware MC step effectively takes 52 ns. Here we explain the estimate for our parameter settings, where replica exchanges are averaged into the effective step time.

The DAU lacks a termination condition, i.e., it will run for the designated number of MC updates, regardless of whether it has already found the planted solution. Thus, we cannot directly extract the TTS based on the returned total 'anneal_time'. We instead use the returned MC updates to solution $\mathrm{updates}_{\mathrm{MC}}$ to determine the t_0 defined in methods under DAU data collection, which subsequently gives $p_{\mathrm{S}}(t_{\mathrm{f}})$. The needed connecting factor is the runtime per MC update t_{MC}, such that $\mathrm{updates}_{\mathrm{MC}} * t_{\mathrm{MC}} = t_0$. Many runs were performed for a total number of MC updates at intervals of repex_interval = 2500, then post-selected only for runs which did not reach the solution state and averaged. A line fit to this data has a slope of effective t_{MC}. However, there are two distinct values of t_{MC} as illustrated for a representative case in figure 5. For the first 100 000 MC steps, $t_{\mathrm{MC}} \approx 3 \times 10^{-6}$ s, then for all subsequent MC updates $t_{\mathrm{MC}} \approx 52 \times 10^{-8}$ s. This finding is consistent regardless of problem size n, and derives from the use of automatic parameter adjustment on a CPU for the first 100 000 MC steps. To find the scaling associated with purely the algorithm, we use $t_{\mathrm{MC}} = 52$ ns for calculating TTS in the main text.

Appendix E. D-Wave detailed data collection

We detail the procedure used in our study for data collection from the DWA. We used the Ising form of the 3R3X instances rather than the QUBO form.

(a) Use Minorminer.find_embedding with the following parameter setting: (max_no_improvement = 100, chainlength_patience = 100, tries = 100, timeout = 1000 (s)), to find and store *per instance*embedding.

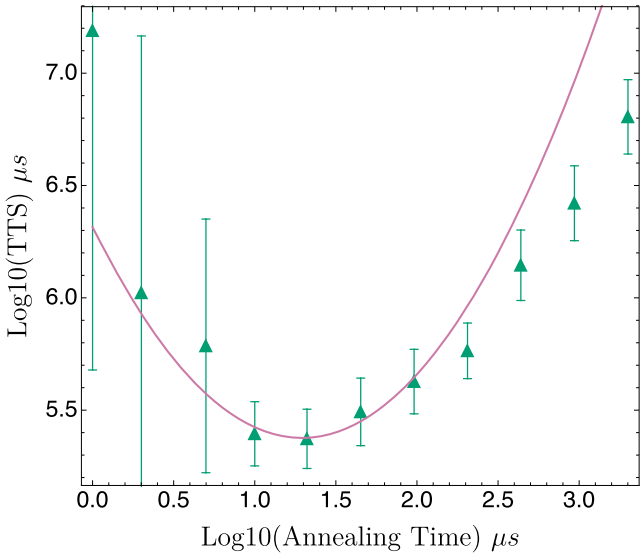

Figure 6. An example of fitting the TTS vs annealing time for $n = 64$ to find optTTS and the optimal annealing time for our DWA analysis.

(b) Generate and save at least 50 gauges of each Ising problem.

(c) Choose a subset of 10 instances and 6 gauges at each N to run for log-uniform distribution of 11 annealing times from 1 μs to 2000 μs and the following set of chain strengths: (2, 1.5, 1).

(d) Randomize the instance/gauge list.

(e) For each instance/gauge:

1. Load the corresponding embedding, use VirtualGraphComposite
2. Manually rescale the Ising problem and its ground state (GS) energy to exactly fit the bias/coupler ranges of the solver.
3. Randomize chain strength list and annealing time list.
4. For each chain strength/annealing time combination:

   (i) Sample 250–500 times, depending on n.

   (ii) Store the full set of raw data.

   (iii) Use majority vote decoding to 'fix' broken chains, store percentage of broken chains.

   (iv) Store a lightly processed data set with number of successes derived from comparison with scaled GS energy.

(f) For each n and chain_strength set of data perform an optTTS analysis over annealing times that bootstraps over instances/gauges in the following way at each annealing time:

1. For each of $\geqslant$1000 bootstraps do:

   (i) Resample Iinstances

   (ii) For each instance:

   (A) Set numSuccess = numFailures = 0.

   (B) Resample Ggauges

   (C) For each gauge add the number of successful runs (that found the GS) to numSuccess, and add the number of failed runs to numFailures (do not sample $p_S$here).

   (iii) Sample p_S from the following distribution: β(numSuccess + 0.5, numFailures + 0.5), store in $p_S$vector.

2. Calculate p_{Sq} here by finding the qth percentile p_S of the current instance's p_S vector of data, store p_{Sq}.

(g) After all $\geqslant$1000 bootstraps are complete, store the mean/standard deviation of across p_{Sq}'s in the p_{Sq} vector over instances to produce the mean TTS at that annealing time.

(h) For each n, chain_strength combination, estimate the minimum TTS across annealing times via a fitting function, as illustrated in figure 6.

(i) Finally, select the best chain_strength value at each n and plot in the final comparison with other solvers.

Appendix F. Toshiba SBM origins

Background. Networks of Kerr parametric oscillators (KPOs) were proposed recently as a method of quantum adiabatic optimization (QAO) for Ising/QUBO problems [91, 92]. Following Goto [91], consider the Hamiltonian

$$H_q(t) = \hbar \sum_{i=1}^{n} \left[\frac{K}{2} a_i^{\dagger 2} a_i^2 - \frac{p(t)}{2} \left(a_i^{\dagger 2} + a_i^2 \right) + \Delta_i a_i^\dagger a_i \right] - \hbar \xi_0 \sum_{i<j}^{n} J_{ij} a_i^\dagger a_j \ , \tag{F1}$$

where $\hbar$ is the reduced Planck constant, $a_i^\dagger / a_i$ is the creation/annihilation operator for the ith oscillator, K is a positive Kerr coefficient, $p(t)$ is the time-dependent parametric two-photon pumping amplitude, Δ_i is the positive detuning frequency between the resonance frequency of the ith oscillator and half the pumping frequency, and ξ_0 is a positive constant with dimensions of frequency. Each KPO is initially set to the vacuum state, $p(0)$ is set to 0, and ξ_0 is set small enough that the vacuum state is the ground state of the initial Hamiltonian. If by the final time t_f the pumping amplitude is sufficiently large ($p(t_f) \gg \max_i \Delta_i$), then each KPO reaches a coherent state with a positive or negative amplitude via a quantum adiabatic bifurcation. If $p(t)$ is increased slowly enough, the quantum adiabatic theorem guarantees the sign of the final amplitude for the ith KPO provides the ith Ising spin of the ground state of the Ising problem [91].

Algorithm. While a physical realization of a network of quantum KPOs for QAO has yet to be realized, a new classical algorithm for Ising/QUBO problems derived from a classical approximation of equation (F1) has been proposed and demonstrated [35]. By approximating $a_j = x_j + \mathrm{i} y_j$, a classical Hamiltonian and derived equations of motion are obtained. Based on empirical observations [28, 91] of the y_j amplitude oscillating around 0 and the need for fast numerical simulation, the authors remove some terms linear in y_j from the equations of motion. Assuming all detunings have the same value Δ, they obtain the final equations used in the simulated bifurcation (SB) algorithm:

$$\dot{x}_i = \Delta y_i \tag{F2a}$$

$$\dot{y}_i = -\left[K x_i^2 - p(t) + \Delta \right] x_i + \xi_0 \sum_{j=1}^{N} J_{ij} x_j \ . \tag{F2b}$$

The SB algorithm is as follows. Use the constant values determined by the KPO-AQO method with the normalization $K = \Delta = 1$ and set all initial x_i and y_i around zero. Gradually increasing $p(t)$ from zero, solve equations (F2a) and (F2b) numerically by a modified explicit symplectic Euler method (see methods in [35]). The sign of the final x_i provides that of the ith spin of an approximate solution of the Ising problem.

ORCID iDs

Tameem Albash https://orcid.org/0000-0003-3916-3985
Itay Hen https://orcid.org/0000-0002-7009-7739
Daniel A Lidar https://orcid.org/0000-0002-1671-1515

References

[1] Wolsey L A and Nemhauser G L 1999 *Integer and Combinatorial Optimization* (New York: Wiley)
[2] Papadimitriou C H and Steiglitz K 2013 *Combinatorial Optimization: Algorithms and Complexity* (*Dover Books on Computer Science*) (New York: Dover)
[3] Matsubara S *et al* 2018 Ising-model optimizer with parallel-trial bit-sieve engine *Complex, Intelligent, and Software Intensive Systems* ed L Barolli and O Terzo (Berlin: Springer) pp 432–8
[4] Tsukamoto S, Takatsu M, Matsubara S and Tamura H 2017 An accelerator architecture for combinatorial optimization problems *Fujitsu Sci. Tech. J.* **53** 8–13
[5] Kaminsky W M and Lloyd S 2004 Scalable architecture for adiabatic quantum computing of NP-hard problems *Quantum Computing and Quantum Bits in Mesoscopic Systems* ed A J Leggett, B Ruggiero and P Silvestrini (Berlin: Springer) pp 229–36
[6] Johnson M W *et al* 2010 A scalable control system for a superconducting adiabatic quantum optimization processor *Supercond. Sci. Technol.* **23** 065004
[7] Berkley A J *et al* 2010 A scalable readout system for a superconducting adiabatic quantum optimization system *Supercond. Sci. Technol.* **23** 105014

[8] Harris R *et al* 2010 Experimental investigation of an eight-qubit unit cell in a superconducting optimization processor *Phys. Rev.* B **82** 024511
[9] Johnson M W *et al* 2011 Quantum annealing with manufactured spins *Nature* **473** 194–8
[10] Bunyk P I *et al* 2014 Architectural considerations in the design of a superconducting quantum annealing processor *IEEE Trans. Appl. Supercond.* **24** 1–10
[11] Weber S J *et al* 2017 Coherent coupled qubits for quantum annealing *Phys. Rev. Appl.* **8** 014004
[12] Novikov S, Hinkey R, Disseler S, Basham J I, Albash T, Risinger A, Ferguson D, Lidar D A and Zick K M 2018 Exploring more-coherent quantum annealing *2018 IEEE Int. Conf. Rebooting Computing (ICRC)* pp 1–7
[13] Grover J A *et al* 2020 Fast, lifetime-preserving readout for high-coherence quantum annealers *PRX Quantum* **1** 020314
[14] McMahon P L *et al* 2016 A fully programmable 100-spin coherent Ising machine with all-to-all connections *Science* **354** 614–7
[15] Yamamoto Y, Aihara K, Leleu T, Kawarabayashi K-i, Kako S, Fejer M, Inoue K and Takesue H 2017 Coherent Ising machines-optical neural networks operating at the quantum limit *npj Quantum Inf.* **3** 49
[16] Ng E, Onodera T, Kako S, McMahon P L, Mabuchi H and Yamamoto Y 2021 Efficient sampling of ground and low-energy Ising spin configurations with a coherent Ising machine (arXiv:2103.05629)
[17] Traversa F L, Ramella C, Bonani F and Di Ventra M 2015 Memcomputing NP-complete problems in polynomial time using polynomial resources and collective states *Sci. Adv.* **1** e1500031
[18] Traversa F L and Di Ventra M 2018 MemComputing integer linear programming (arXiv:1808.09999)
[19] Di Ventra M and Traversa F L 2018 Perspective: memcomputing: leveraging memory and physics to compute efficiently *J. Appl. Phys.* **123** 180901
[20] Yamamoto K *et al* 2021 STATICA: a 512-spin 0.25 M-weight annealing processor with an all-spin-updates-at-once architecture for combinatorial optimization with complete spin–spin interactions *IEEE J. Solid State Circ.* **56** 165–78
[21] Rønnow T F, Wang Z, Job J, Boixo S, Isakov S V, Wecker D, Martinis J M, Lidar D A and Troyer M 2014 Defining and detecting quantum speedup *Science* **345** 420–4
[22] Hen I, Job J, Albash T, Ronnow T F, Troyer M and Lidar D A 2015 Probing for quantum speedup in spin-glass problems with planted solutions *Phys. Rev.* A **92** 042325
[23] Mandrà S, Zheng Z, Wang W, Perdomo-Ortiz A and Katzgraber H G 2016 Strengths and weaknesses of weak-strong cluster problems: a detailed overview of state-of-the-art classical heuristics versus quantum approaches *Phys. Rev.* A **94** 022337
[24] Albash T and Daniel A 2018 Lidar demonstration of a scaling advantage for a quantum annealer over simulated annealing *Phys. Rev.* X **8** 031016
[25] Ryan H *et al* 2019 Experimental investigation of performance differences between coherent Ising machines and a quantum annealer *Sci. Adv.* **5** eaau0823
[26] Aramon M, Rosenberg G, Valiante E, Miyazawa T, Tamura H and Katzgraber H G 2019 Physics-inspired optimization for quadratic unconstrained problems using a digital annealer *Front. Phys.* **7** 48
[27] Matsubara S, Takatsu M, Miyazawa T, Shibasaki T, Watanabe Y, Takemoto K and Tamura H 2020 Digital annealer for high-speed solving of combinatorial optimization problems and its applications *2020 25th Asia and South Pacific Design Automation Conf. (ASP-DAC)* pp 667–72
[28] Goto H 2019 Quantum computation based on quantum adiabatic bifurcations of Kerr-nonlinear parametric oscillators *J. Phys. Soc. Japan* **88** 061015
[29] Perdomo-Ortiz A *et al* 2019 Readiness of quantum optimization machines for industrial applications *Phys. Rev. Appl.* **12** 014004
[30] Şeker O, Neda T and Bodur M 2020 Digital annealer for quadratic unconstrained binary optimization: a comparative performance analysis (arXiv:2012.12264)
[31] Aiken J and Traversa F L 2020 Memcomputing for accelerated optimization (arXiv:2003.10644)
[32] King A D and Bernoudy W 2020 Performance benefits of increased qubit connectivity in quantum annealing three-dimensional spin glasses (arXiv:2009.12479)
[33] Goto H, Endo K, Suzuki M, Sakai Y, Kanao T, Hamakawa Y, Hidaka R, Yamasaki M and Tatsumura K 2021 High-performance combinatorial optimization based on classical mechanics *Sci. Adv.* **7** eabe7953
[34] Gonzalez Calaza C D, Willsch D and Michielsen K 2021 Garden optimization problems for benchmarking quantum annealers (arXiv:2101.10827)
[35] Goto H, Tatsumura K and Dixon A R 2019 Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems *Sci. Adv.* **5** eaav2372
[36] Kelly B, Paul B, Raymond J and Roy A 2020 Next-generation topology of D-wave quantum processors (arXiv:2003.00133)
[37] Bernaschi M, Bisson M, Fatica M, Marinari E, Martin-Mayor V, Parisi G and Ricci-Tersenghi F 2021 How we are leading a 3-XORSAT challenge: from the energy landscape to the algorithm and its efficient implementation on GPUs (a) *Europhys. Lett.* **133** 60005
[38] Swendsen R H and Wang J-S 1986 Replica Monte Carlo simulation of spin-glasses *Phys. Rev. Lett.* **57** 2607–9
[39] Geyer C J 1991 Parallel tempering *Computing Science and Statistics Proc. 23rd Symp. Interface* ed E M Keramidas and S M Kaufman (New York: American Statistical Association) p 156
[40] Hukushima K and Nemoto K 1996 Exchange Monte Carlo method and application to spin glass simulations *J. Phys. Soc. Japan* **65** 1604–8
[41] Hestenes M R and Stiefel E 1952 Methods of conjugate gradients for solving linear systems *J. Res. Natl Bur. Stand.* **49** 409–36
[42] Franz S, Mézard M, Ricci-Tersenghi F, Weigt M and Zecchina R 2001 A ferromagnet with a glass transition *Europhys. Lett.* **55** 465–71
[43] Barthel W, Hartmann A K, Leone M, Ricci-Tersenghi F, Weigt M and Zecchina R 2002 Hiding solutions in random satisfiability problems: a statistical mechanics approach *Phys. Rev. Lett.* **88** 188701
[44] Jörg T, Krzakala F, Semerjian G and Zamponi F 2010 First-order transitions and the performance of quantum algorithms in random optimization problems *Phys. Rev. Lett.* **104** 207206
[45] Ricci-Tersenghi F 2010 Being glassy without being hard to solve *Science* **330** 1639–40
[46] Guidetti M and Young A P 2011 Complexity of several constraint-satisfaction problems using the heuristic classical algorithm walksat *Phys. Rev.* E **84** 011102
[47] Farhi E, Gosset D, Hen I, Sandvik A W, Shor P, Young A P and Zamponi F 2012 Performance of the quantum adiabatic algorithm on random instances of two optimization problems on regular hypergraphs *Phys. Rev.* A **86** 052334
[48] Hen I 2019 Equation planting: a tool for benchmarking Ising machines *Phys. Rev. Appl.* **12** 011003

[49] Valiante E, Hernandez M, Amin B and Katzgraber H G 2020 Scaling overhead of locality reduction in binary optimization problems (arXiv:2012.09681)
[50] Walter V and Daniel A 2016 Lidar optimally stopped optimization *Phys. Rev. Appl.* **6** 054016
[51] AWS Amazon https://aws.amazon.com/marketplace/pp/prodview-f3hbaz4q3y32y (accessed 20 August 2020)
[52] Metropolis N, Rosenbluth A W, Rosenbluth M N, Teller A H and Teller E 1953 Equation of state calculations by fast computing machines *J. Chem. Phys.* **21** 1087–92
[53] Hastings W K 1970 Monte Carlo sampling methods using Markov chains and their applications *Biometrika* **57** 97–109
[54] Katzgraber H G, Trebst S, Huse D A and Troyer M 2006 Feedback-optimized parallel tempering Monte Carlo *J. Stat. Mech.* P03018
[55] Zheng Z, Fang C and Katzgraber H G 2020 Borealis—a generalized global update algorithm for Boolean optimization problems *Optim. Lett.* **14** 2495–514
[56] Yates C A and Klingbeil Y 2013 Recycling random numbers in the stochastic simulation algorithm *J. Chem. Phys.* **138** 094103
[57] Houdayer J 2001 A cluster Monte Carlo algorithm for two-dimensional spin glasses *Eur. Phys. J.* B **22** 479–84
[58] Zheng Z, Ochoa A J and Katzgraber H G 2015 Efficient cluster algorithm for spin glasses in any space dimension *Phys. Rev. Lett.* **115** 077201
[59] Peters E A J F and de With G 2012 Rejection-free Monte Carlo sampling for general potentials *Phys. Rev.* E **85** 026703
[60] Kadowaki T and Nishimori H 1998 Quantum annealing in the transverse Ising model *Phys. Rev.* E **58** 5355–63
[61] Brooke J, Bitko D, Rosenbaum G and Aeppli G 1999 Quantum annealing of a disordered magnet *Science* **284** 779–81
[62] Brooke J, Rosenbaum T F and Aeppli G 2001 Tunable quantum tunnelling of magnetic domain walls *Nature* **413** 610–3
[63] Farhi E, Goldstone J, Gutmann S, Lapan J, Lundgren A and Preda D 2001 A quantum adiabatic evolution algorithm applied to random instances of an NP-complete problem *Science* **292** 472–5
[64] Choi V 2008 Minor-embedding in adiabatic quantum computation: I. The parameter setting problem *Quantum Inf. Process.* **7** 193–209
[65] Choi V 2011 Minor-embedding in adiabatic quantum computation: II. Minor-universal graph design *Quantum Inf. Process.* **10** 343–53
[66] Bellitti M, Ricci-Tersenghi F and Scardicchio A 2021 Entropic barriers as a reason for hardness in both classical and quantum algorithms (arXiv:2102.00182)
[67] Jansen S, Ruskai M-B and Seiler R 2007 Bounds for the adiabatic approximation with applications to quantum computation *J. Math. Phys.* **48** 102111
[68] Amin M H 2015 Searching for quantum speedup in quasistatic quantum annealers *Phys. Rev.* A **92** 052323
[69] Boixo S *et al* 2016 Computational multiqubit tunnelling in programmable quantum annealers *Nat. Commun.* **7** 10327
[70] Albash T, Martin-Mayor V and Hen I 2017 Temperature scaling law for quantum annealing optimizers *Phys. Rev. Lett.* **119** 110502
[71] Dickson N G *et al* 2013 Thermally assisted quantum annealing of a 16-qubit problem *Nat. Commun.* **4** 1903
[72] Albash T, Walter V, Mishra A, Warburton P A and Lidar D A 2015 Consistency tests of classical and quantum models for a quantum annealer *Phys. Rev.* A **91** 042314
[73] *D-Wave: Technical Description of the QPU* https://docs.dwavesys.com/docs/latest/doc_qpu.html (accessed 31 October 2020)
[74] Zheng Z, Ochoa A J, Schnabel S, Hamze F and Katzgraber H G 2016 Best-case performance of quantum annealers on native spin-glass benchmarks: how chaos can affect success probabilities *Phys. Rev.* A **93** 012317
[75] Albash T, Martin-Mayor V and Hen I 2019 Analog errors in Ising machines *Quantum Sci. Technol.* **4** 02LT03
[76] Pearson A, Mishra A, Hen I and Lidar D A 2019 Analog errors in quantum annealing: doom and hope *npj Quantum Inf.* **5** 107
[77] Roland J and Cerf N J 2002 Quantum search by local adiabatic evolution *Phys. Rev.* A **65** 042308
[78] Albash T and Daniel A 2018 Lidar adiabatic quantum computation *Rev. Mod. Phys.* **90** 015002
[79] Crosson E J and Lidar D A 2020 Prospects for quantum enhancement with diabatic quantum annealing (arXiv:2008.09913)
[80] Job J and Lidar D 2018 Test-driving 1000 qubits *Quantum Sci. Technol.* **3** 030501
[81] Job J 2018 The theory and practice of benchmarking quantum annealers *PhD Thesis* University of Southern California, Los Angeles, CA
[82] Rosenthal J S, Dote A, Dabiri K, Tamura H, Chen S and Sheikholeslami A 2020 Jump Markov chains and rejection-free metropolis algorithms (arXiv:1910.13316 [math.ST])
[83] Lipowski A and Lipowska D 2012 Roulette-wheel selection via stochastic acceptance *Physica* A **391** 2193–6
[84] Hukushima K 1999 Domain-wall free energy of spin-glass models: numerical method and boundary conditions *Phys. Rev.* E **60** 3606–13
[85] Klymko C, Sullivan B D and Humble T S 2014 Adiabatic quantum programming: minor embedding with hard faults *Quantum Inf. Process.* **13** 709–29
[86] Marshall J, Di Gioacchino A and Rieffel E G 2020 Perils of embedding for sampling problems *Phys. Rev. Res.* **2** 023020
[87] Boothby T, King A D and Roy A 2016 Fast clique minor generation in Chimera qubit connectivity graphs *Quantum Inf. Process.* **15** 495–508
[88] Zbinden S, Bärtschi A, Djidjev H and Eidenbenz S 2020 Embedding algorithms for quantum annealers with chimera and pegasus connection topologies *High Performance Computing: 35th Int. Conf., ISC High Performance 2020* vol 12151 (Frankfurt/Main, Germany June 22–25, 2020) pp 187–206
[89] Cai J, Macready W G and Roy A 2014 A practical heuristic for finding graph minors (arXiv:1406.2741)
[90] Wolfram Research Inc. 2020 *Mathematica, Version 12.2* (https://www.wolfram.com/mathematica)
[91] Goto H 2016 Bifurcation-based adiabatic quantum computation with a nonlinear oscillator network *Sci. Rep.* **6** 21686
[92] Puri S, Andersen C K, Grimsmo A L and Blais A 2017 Quantum annealing with all-to-all connected nonlinear oscillators *Nat. Commun.* **8** 15785